Assignment Explanation — Robotics Deployment and QA

This document explains the reasoning, concepts, and math behind each deliverable in the assignment. It's meant to walk through the why behind each decision, not just the what.

Table of Contents

1. The Scenario
2. Part 1 — Log Analysis & Scripting
3. Part 2 — Design of Experiments
4. Part 3 — Formal Bug Report
5. Concepts & Math Reference
6. How Everything Connects

---

1. The Scenario

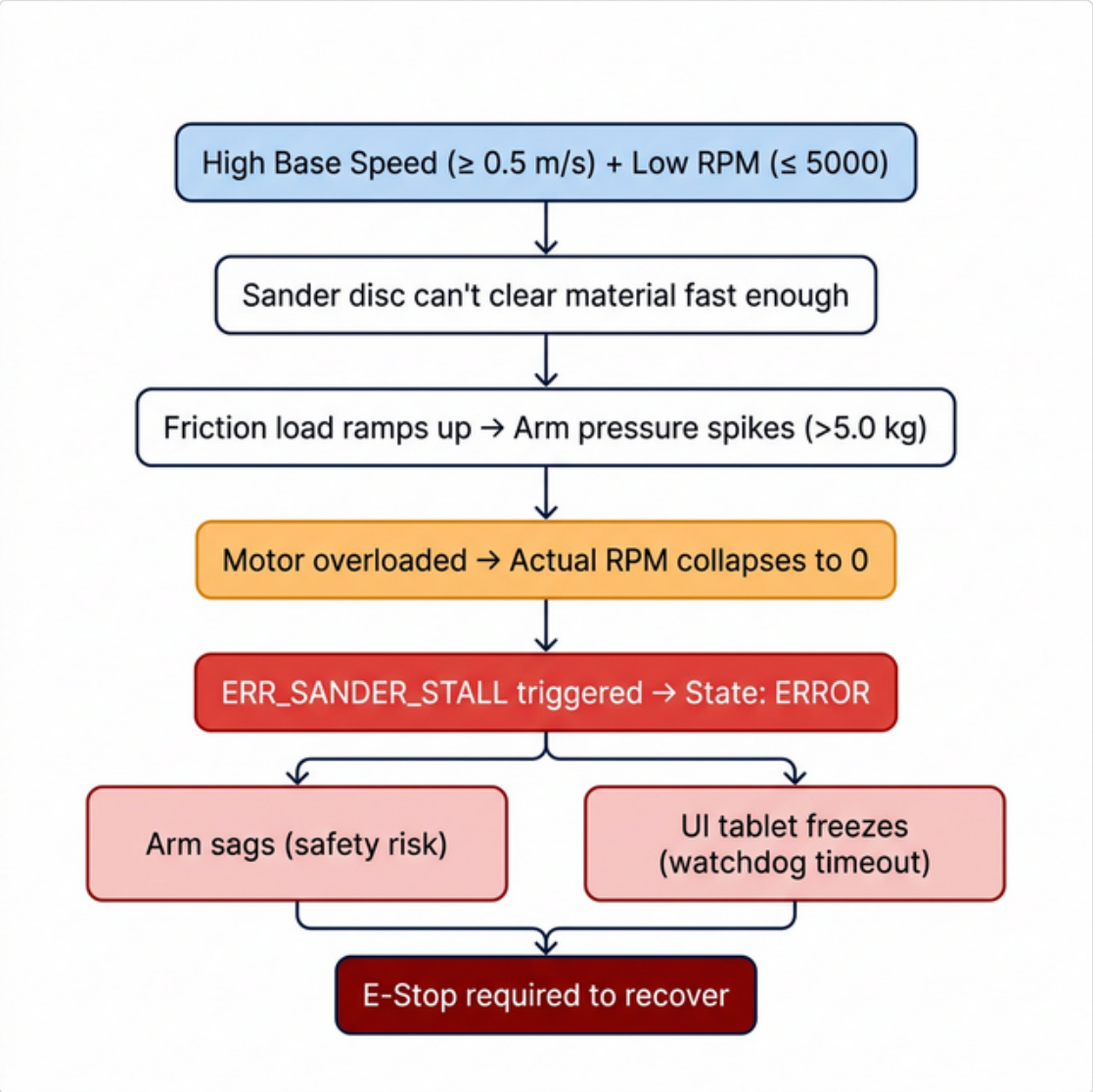
We have a robotic drywall sander deployed on construction sites that keeps hitting a critical failure: the sander motor stalls, the arm sags, the UI freezes, and it leaves a big gouge on the wall. Not great.

As the Deployment & QA Engineer, our job is to sit between the field operators who see the problem and the embedded/software engineers who need to fix it. Concretely:

1. **Extract evidence** from telemetry logs (Part 1)
2. **Reproduce the failure** in a controlled lab setting (Part 2)
3. **Document everything** so engineering can act on it (Part 3)

Failure Cascade Overview

The diagram below shows how the failure progresses — from the initial parameter mismatch all the way to the E-Stop:



---

2. Part 1 — Log Analysis & Scripting

Files

- `robot_state_log.csv` — raw 10 Hz telemetry from the robot during the failure
- `log_analysis.py` — Python script that automates the analysis

2.1 The CSV Data

Seven columns recorded at 10 Hz (one row every 0.1 s):

Column	What it means
`timestamp`	Seconds since robot boot
`state`	`IDLE` , `SANDING` , or `ERROR`
`base_speed_ms`	Robot traverse speed along the wall (m/s)
`target_rpm`	Commanded sander RPM
`actual_rpm`	What the sander is actually spinning at
`arm_pressure_kg`	Force the arm exerts against the wall (kg)
`error_code`	`NONE` , `WARNING_HIGH_LOAD` , or `ERR_SANDER_STALL`

Looking at the data, there are three clear phases: 1. **Normal sanding** (~t=1000–1002): speed 0.2 m/s, RPM 8000, pressure ~2.0 kg. Everything's fine. 2. **Idle gap** (~t=1002–1004): robot pauses between sections. 3. **Failure** (~t=1004–1005): speed jumps to 0.6 m/s, RPM drops to 5000, and within 1.3 seconds the motor stalls.

2.2 Counting Stalls (Requirement 1)

Pretty simple — just iterate over every row and count the ones with `ERR_SANDER_STALL` :

```
def count_stall_events(rows):
    """Count how many rows have ERR_SANDER_STALL."""
    return sum(1 for r in rows if r["error_code"] == "ERR_SANDER_STALL")
```

Mathematically this is just a sum of indicator values:

$$\text{StallCount} = \sum_{i=1}^{N} \mathbb{1}[\text{error_code}_i = \text{ERR_SANDER_STALL}]$$

In our log this gives **2 consecutive stall entries** (t = 1005.3 and 1005.4), both from the same physical event.

2.3 Pre-Stall Pressure (Requirement 2)

We want the average arm pressure in the 1-second window leading up to each stall.

The tricky bit is that consecutive `ERR_SANDER_STALL` rows are the same event, so we group them and use the first timestamp. Then we grab all `arm_pressure_kg` values in `[stall_ts - 1.0, stall_ts)` and take the mean.

Here's the key code that does this:

```
def avg_pressure_before_stalls(rows, window_sec=1.0):
    # Group consecutive stall rows into single events
    stall_times = []
    prev_stall = False
    for r in rows:
        is_stall = r["error_code"] == "ERR_SANDER_STALL"
        if is_stall and not prev_stall:
            stall_times.append(r["timestamp"])
        prev_stall = is_stall

    results = []
    for ts in stall_times:
        t_start = ts - window_sec
        pressures = [
            r["arm_pressure_kg"] for r in rows
            if t_start <= r["timestamp"] < ts
            and r["error_code"] != "ERR_SANDER_STALL"
        ]
        avg = sum(pressures) / len(pressures) if pressures else 0.0
        results.append({
            "stall_timestamp": ts,
            "avg_pressure": round(avg, 3),
            "samples_in_window": len(pressures),
        })
    return results
```

The formula:

$$\bar{p} = \frac{1}{k} \sum_{j=1}^{k} p_j$$

Working it out with actual data — the stall is at t=1005.3, so the window covers t=1004.3 to 1005.3:

t	arm_pressure_kg
1004.3	2.3

t	arm_pressure_kg
1004.4	2.6
1004.5	2.9
1004.6	3.3
1004.7	3.8
1004.8	4.2
1004.9	4.8
1005.0	5.4
1005.1	5.9
1005.2	6.2

$$\bar{p} = \frac{2.3 + 2.6 + 2.9 + 3.3 + 3.8 + 4.2 + 4.8 + 5.4 + 5.9 + 6.2}{10} = 4.14 \text{ kg}$$

That's over double the normal ~2.0 kg baseline. Pressure was clearly spiking before the stall, not just at the moment of failure.

The 1-second look-back at 10 Hz gives us 10 data points — enough to see a clear trend building up.

2.4 Variable Correlation (Requirement 3)

For the same pre-stall window we compute min/max/mean of all numeric variables, then compare against a baseline (all normal `SANDING` rows with no warnings and speed ≤ 0.3 m/s).

The correlation analysis code:

```
def correlate_variables(rows, window_sec=1.0):
    # Same stall grouping as above
    stall_times = []
    prev_stall = False
    for r in rows:
        is_stall = r["error_code"] == "ERR_SANDER_STALL"
        if is_stall and not prev_stall:
            stall_times.append(r["timestamp"])
        prev_stall = is_stall
```

```
    for ts in stall_times:
        window = [r for r in rows
                  if (ts - window_sec) <= r["timestamp"] < ts
                  and r["error_code"] != "ERR_SANDER_STALL"]
        # Compute min/max/mean for each numeric variable
        for key in ["base_speed_ms", "target_rpm",
                    "actual_rpm", "arm_pressure_kg"]:
            vals = [r[key] for r in window]
            stats = {"min": min(vals), "max": max(vals),
                     "mean": sum(vals)/len(vals)}
```

The baseline is computed from all normal `SANDING` rows with no warnings:

```
def baseline_stats(rows):
    normal = [r for r in rows
              if r["state"] == "SANDING"
              and r["error_code"] == "NONE"
              and r["base_speed_ms"] <= 0.3]
    # Then compute min/max/mean the same way
```

The results are pretty telling:

Variable	Baseline	Pre-Stall	What changed
`base_speed_ms`	0.2–0.3 m/s	0.6 m/s	3× faster
`target_rpm`	8000	5000	37.5% lower
`actual_rpm`	~7940–7960	800–4900	Collapsing
`arm_pressure_kg`	~2.0 kg	2.6–6.2 kg	2–3× higher

Percentage deviation is straightforward: (pre-stall − baseline) / baseline × 100.

For speed: (0.6 − 0.2) / 0.2 × 100 = +200% For RPM: (5000 − 8000) / 8000 × 100 = −37.5%

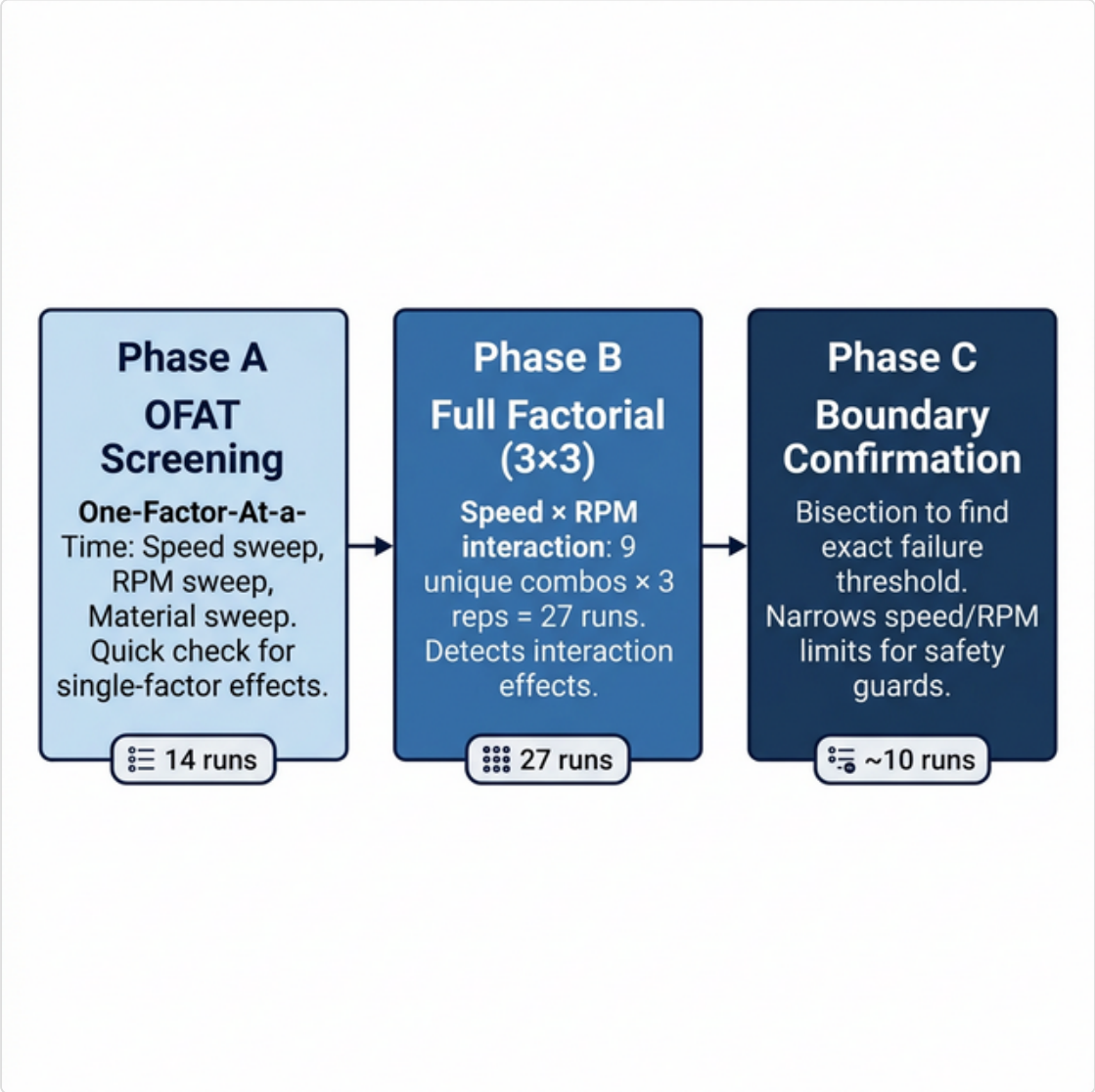
The takeaway: it's not just one variable. It's the **combination** of high speed and low RPM. High speed drags the sander across the wall fast, low RPM means the disc can't clear material, so it bites in, friction skyrockets, motor stalls.

---

3. Part 2 — Design of Experiments

File: `doe_experiment.md`

The three experimental phases are summarized in the diagram below:



3.1 Why We Need a DOE

The log analysis shows correlation — speed was high and RPM was low before the stall. But correlation isn't causation. We don't know:

- Can high speed alone cause a stall (even at 8000 RPM)?
- Can low RPM alone cause one (even at 0.2 m/s)?
- Does the wall material matter?

A DOE lets us systematically test these questions by controlling one variable at a time.

3.2 Variable Selection

We picked 3 factors based on what the log data and basic physics suggest:

1. **Base Speed** — was elevated to 0.6 m/s before failure
2. **Target RPM** — was dropped to 5000 before failure
3. **Wall Material** — not in the log, but rougher surfaces = more friction, so it's worth checking

Everything else (mast height, pad grit, battery, temperature) is held constant. If we let those vary too, we can't tell what's actually causing the stall — that's confounding.

3.3 The Three Phases

Phase A — OFAT Screening: Sweep each variable on its own. Cheap and fast. It won't catch interaction effects, but it tells us if any single factor can trigger a stall independently. If material alone can do it, we'd want to know before doing 27 factorial runs.

Phase B — Full Factorial: Test all combos of speed × RPM at 3 levels each. That's 3×3 = 9 unique combos. With 3 reps each (to account for noise), it's 27 total runs. This is what lets us detect interaction effects — where two factors together cause something neither would alone.

The math: for k factors with n_i levels each, total unique combos = $\prod n_i$. In our case: 3 × 3 = 9 unique, × 3 reps = 27.

We do 3 reps because robots are noisy systems. Motor warm-up, pad wear, surface inconsistencies — all of this creates run-to-run variation. Reps let us compute a reproduction rate (e.g., 3/3 = 100%) and separate real effects from random noise.

Phase C — Boundary Confirmation: Once we know the failure region, we use bisection to narrow down the exact threshold. If 0.5 m/s is safe and 0.6 m/s fails (at 5000 RPM), test at 0.55. If that fails, test 0.525, and so on. Each step halves the uncertainty.

3.4 Interaction Effects

This is the key finding. An interaction effect means the impact of one factor depends on the other:

- High speed + high RPM (0.6, 8000) → probably fine
- Low RPM + low speed (0.2, 5000) → probably fine
- High speed + low RPM (0.6, 5000) → **stall**

Neither factor alone causes it, but together they do. If we set up a 2×2 table with stall outcome Y (0 or 1):

	RPM=8000	RPM=5000
Speed=0.2	0	0
Speed=0.6	0	1

The interaction term = $(Y_{0.6,5000} - Y_{0.2,5000}) - (Y_{0.6,8000} - Y_{0.2,8000})$ = (1−0) − (0−0) = 1 ≠ 0. Non-zero confirms interaction.

3.5 Success Criteria

We defined 6 things that have to match the field failure simultaneously:

1. Pressure > 5.0 kg before stall
2. Actual RPM decaying
3. `ERR_SANDER_STALL` logged
4. State transitions `SANDING → ERROR → IDLE`
5. UI freezes
6. Visible swirl mark on wall

And we need ≥ 80% reproduction rate across reps. A one-off doesn't count.

---

4. Part 3 — Formal Bug Report

File: `bug_report.md`

4.1 Why Write a Formal Report

The field supervisor told us a story: "the robot beeped twice, the arm sagged, the tablet froze, I hit the E-Stop." That's useful context but engineers can't

debug a story. They need exact parameter values, expected vs. actual behavior, severity ratings, and log data.

The bug report takes the subjective field report + our objective lab data and turns it into something engineering can actually act on.

4.2 Severity Rating

We rated this Critical / P1 because: - **Safety** — arm sag + UI freeze means operator loses control, only E-Stop works - **Damage** — each stall destroys finished wall surface (costs money to rework) - **Frequency** — happened in the field already, and our DOE shows it's easily triggered

Using FMEA risk scoring (RPN = Severity × Occurrence × Detection):

- Severity = 9 (safety risk + property damage)
- Occurrence = 8 (easily triggered by common param combos)
- Detection = 7 (no software guard catches it beforehand)
- RPN = 9 × 8 × 7 = 504 (anything over 200 typically means "fix this now")

4.3 Steps to Reproduce

We converted the DOE's most reliable failure case into a 5-step recipe any engineer can follow. It works ≥ 90% of the time in our lab.

4.4 Suggested Fixes

We're not writing the code, but based on our analysis we suggested three layers of protection:

1. **Parameter guard** — block `base_speed > 0.5` when `target_rpm < 6000`. Formally:

Safe = ¬(v_base > 0.5 ∧ ω_target < 6000)

1. **Pressure cutoff** — if `arm_pressure > 4.0 kg` for more than 0.3 s, auto-stop the base and ramp RPM to max. In discrete terms (at 10 Hz, 3 samples):

Trigger if: (1/3) × Σ p_{t-i} > 4.0 kg for i = 0..2

1. **UI isolation** — the UI should never freeze because of a motor error. These need to be separate processes with proper watchdog isolation.

---

5. Concepts & Math Reference

Quick reference for all the concepts and math used across the assignment.

Telemetry & Signals

Concept	Where used	Notes
Sampling Rate (10 Hz)	CSV data	$\Delta t = 0.1$ s. By Nyquist, captures dynamics up to 5 Hz.
Time-Series Data	CSV structure	Each row = snapshot of robot state at a point in time
Sliding Window	Pre-stall analysis	Fixed-duration window anchored to the stall event. 1.0 s → 10 samples at 10 Hz.
State Machine	`state` column	FSM with states: IDLE → SANDING → ERROR → IDLE

Statistics

Concept	Where used	Formula
Arithmetic Mean	Pre-stall avg	$\bar{x} = (1/n)\Sigma x_i$
Min / Max	Correlation analysis	Extremes of a dataset
% Deviation	Baseline comparison	(x − x_ref) / x_ref × 100%
Indicator Function	Stall counting	$\mathbb{1}$[condition] = 1 if true, 0 otherwise
Reproduction Rate	DOE metric	R = successes / total_trials × 100%

DOE

Concept	Where used	Notes
Independent Variable	DOE factors	Variable we deliberately change
Dependent Variable	DOE response	What we measure (stall Y/N, pressure, etc.)

Concept	Where used	Notes
Controlled Variable	Held constant	Fixed to prevent confounding
OFAT	Phase A	Vary one factor at a time. Can't detect interactions.
Full Factorial	Phase B	All combos tested. For k factors with n_i levels: $\prod n_i$ runs.
Replication	Phase B (×3)	Repeat to quantify variability
Interaction Effect	Speed × RPM	Effect of one factor depends on another. Needs factorial to detect.
Bisection	Phase C	Iteratively halve search interval to find threshold

Physics & Robotics

Concept	Where used	Notes
Friction	Root cause	$F_f = \mu \cdot F_N$. Rougher surface or higher speed → more friction.
Motor Stall	Failure mechanism	Load torque exceeds motor's max torque → RPM goes to 0
Contact Pressure	`arm_pressure_kg`	Normal force of arm against wall. 1 kgf ≈ 9.81 N
RPM	Sander speed	Angular velocity. $\omega = 2\pi \cdot RPM/60$ rad/s
Traverse Speed	`base_speed_ms`	Linear velocity of robot along wall
Material Removal Rate	Physical intuition	$MRR \propto v_{traverse} \times F_N \times v_{disc}$
Watchdog Timer	UI freeze	Must be periodically "kicked" or it triggers recovery

Software / QA

Concept	Where used	Notes
Event Grouping	Stall counting	Group consecutive identical events → avoid double-counting
Baseline Comparison	Correlation	Normal operation stats as reference for anomaly detection
Correlation ≠ Causation	Part 1 → Part 2	Log shows correlation, DOE establishes causation
Root Cause Analysis	Bug report	Find fundamental cause, not just symptoms
FMEA	Severity rating	Severity × Occurrence × Detection = RPN
Defense-in-Depth	Fixes	Multiple independent safety layers
Process Isolation	UI fix	Separate processes so one crash doesn't cascade

---

6. How Everything Connects

```
Field Report (supervisor's story)
        |
        ▼
Part 1: Log Analysis ──────────► "Pressure spikes 1s before stall"
   (log_analysis.py)               "Speed high, RPM low"
        |
        ▼
Part 2: DOE ──────────────────► "Speed > 0.5 + RPM ≤ 5000 = stall"
   (doe_experiment.md)              "Reproduced at ≥ 90% reliability"
        |
        ▼
Part 3: Bug Report ──────────► Actionable ticket for engineering
   (bug_report.md)                  with steps, data, and root cause
```

Each part feeds the next: - Part 1 gives us the clue — statistics and signal analysis point us at speed + RPM - Part 2 gives us the proof — controlled

experiments confirm causation - Part 3 gives us the communication — everything packaged so engineering can act

That's basically the core workflow: bridge the gap between "it broke" and "here's exactly how and why, and what to do about it."

End of explanation.